Name : Aditya Bhende

roll no : 28

Class : C-C2

```
import warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score

from scipy.cluster.hierarchy import linkage, dendrogram

sns.set_theme(style='whitegrid')
plt.rcParams["figure.figsize"] = (8, 5)

df = pd.read_csv('Wholesale customers data.csv')
```

```
df.head()
```

	Channel	Region	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen
0	2	3	12669	9656	7561	214	2674	1338
1	2	3	7057	9810	9568	1762	3293	1776
2	2	3	6353	8808	7684	2405	3516	7844
3	1	3	13265	1196	4221	6404	507	1788
4	2	3	22615	5410	7198	3915	1777	5185

```
X = df.select_dtypes(include=[np.number])

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
X_scaled_df = pd.DataFrame(
    X_scaled,
    columns=X.columns,
    index=X.index
)
```

```
display(X_scaled_df.head())
```

	Channel	Region	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen
0	1.448652	0.590668	0.052933	0.523568	-0.041115	-0.589367	-0.043569	-0.066339
1	1.448652	0.590668	-0.391302	0.544458	0.170318	-0.270136	0.086407	0.089151
2	1.448652	0.590668	-0.447029	0.408538	-0.028157	-0.137536	0.133232	2.243293
3	-0.690297	0.590668	0.100111	-0.624020	-0.392977	0.687144	-0.498588	0.093411
4	1.448652	0.590668	0.840239	-0.052396	-0.079356	0.173859	-0.231918	1.299347

```
print("\nScaled feature means (approximately zero):")
print(X_scaled_df.std(ddof=0).round(3))
```

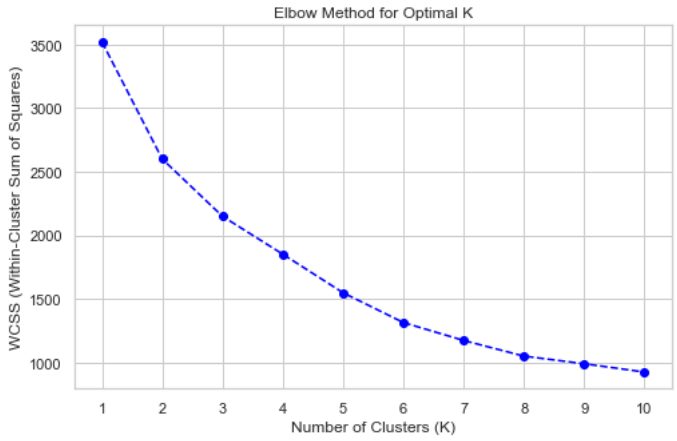
```
Scaled feature means (approximately zero):
Channel             1.0
Region              1.0
```

```
Fresh              1.0
Milk               1.0
Grocery            1.0
Frozen             1.0
Detergents_Paper   1.0
Delicassen         1.0
dtype: float64
```

```
wcss = []
k_values = range(1, 11)

for k in k_values:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)


plt.figure(figsize=(8, 5))
plt.plot(k_values, wcss, marker='o', linestyle='--', color='blue')
plt.title('Elbow Method for Optimal K')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('WCSS (Within-Cluster Sum of Squares)')
plt.xticks(k_values)
plt.grid(True)
plt.show()
```




```
silhouette_values=[]

for k in range(2,11):
    model=KMeans(n_clusters=k,random_state=42,n_init=10)
    labels=model.fit_predict(X_scaled)
    score=silhouette_score(X_scaled,labels)
    silhouette_values.append(score)

silhoutte_table=pd.DataFrame({
    "Number of cluster": range(2,11),
    "Silhouette Score": silhouette_values
})

print(silhoutte_table)
```

```
   Number of cluster  Silhouette Score
0                  2          0.373236
1                  3          0.356769
2                  4          0.347824
3                  5          0.345174
4                  6          0.354873
5                  7          0.363832
6                  8          0.363137
7                  9          0.341903
8                 10          0.360893
```

```
suggested_k=int(
    silhoutte_table.loc[silhoutte_table["Silhouette Score"].idxmax(), "Number of cluster"]
)
```

```
kmeans = KMeans(n_clusters=suggested_k, random_state=42, n_init=10)
kmeans_labels = kmeans.fit_predict(X_scaled)

df_kmeans=df.copy()
df_kmeans["kMeans_cluster"]=kmeans_labels

print(df_kmeans.head())
```

```
   Channel  Region  Fresh  Milk  Grocery  Frozen  Detergents_Paper  \
0        2       3  12669  9656     7561     214              2674
1        2       3   7057  9810     9568    1762              3293
2        2       3   6353  8808     7684    2405              3516
3        1       3  13265  1196     4221    6404               507
4        2       3  22615  5410     7198    3915              1777

   Delicassen  kMeans_cluster
0        1338               0
1        1776               0
2        7844               0
3        1788               1
4        5185               0
```

```
hierachical_model = AgglomerativeClustering(n_clusters=suggested_k, linkage="ward", affinity="euclidean")

hierarchical_labels = hierachical_model.fit_predict(X_scaled)
```

```
df_hierarchical=df.copy()
df_hierarchical["Hierarchical_Cluster"]=hierarchical_labels

print("Hierarchical clustering completed.")
print("\nNumber of customers in each cluster:")
print(df_hierarchical["Hierarchical_Cluster"].value_counts().sort_index())

print(df_hierarchical.head())
```

```
Hierarchical clustering completed.

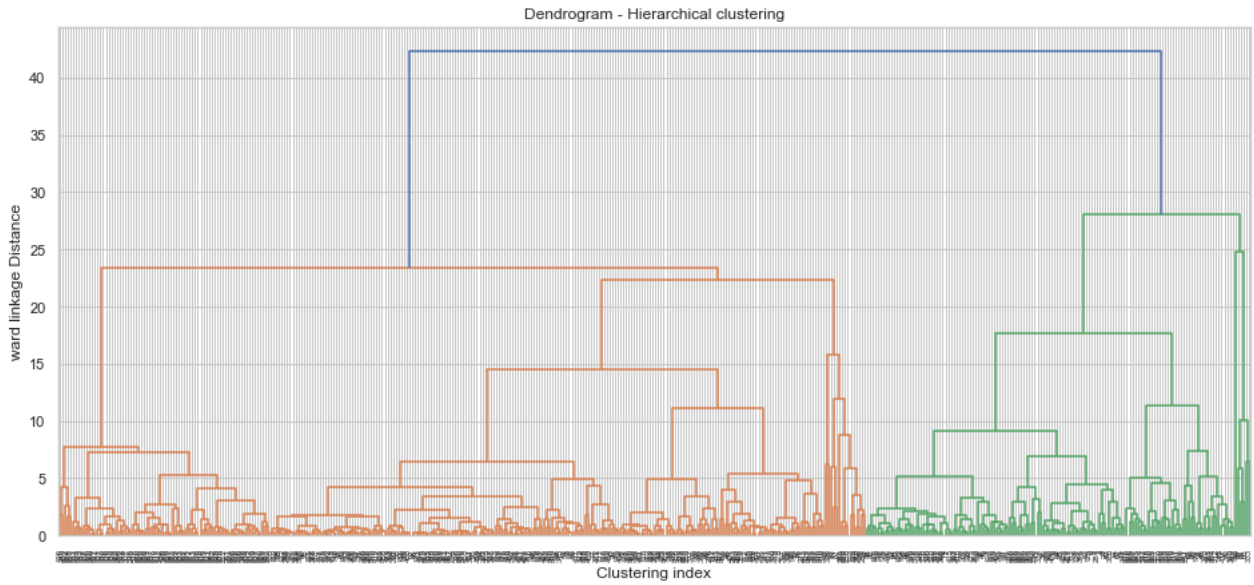
Number of customers in each cluster:
0    142
1    298
Name: Hierarchical_Cluster, dtype: int64
   Channel  Region  Fresh  Milk  Grocery  Frozen  Detergents_Paper  \
0        2       3  12669  9656     7561     214              2674
1        2       3   7057  9810     9568    1762              3293
2        2       3   6353  8808     7684    2405              3516
3        1       3  13265  1196     4221    6404               507
4        2       3  22615  5410     7198    3915              1777

   Delicassen  Hierarchical_Cluster
0        1338                     0
1        1776                     0
2        7844                     0
3        1788                     1
4        5185                     0
```

```
Z=linkage(X_scaled,method="ward",metric="euclidean")

plt.figure(figsize=(16,7))
dendrogram(Z,leaf_rotation=90,leaf_font_size=6)

plt.title("Dendrogram - Hierarchical clustering")
plt.xlabel("Clustering index")
plt.ylabel("ward linkage Distance")
plt.show()
```

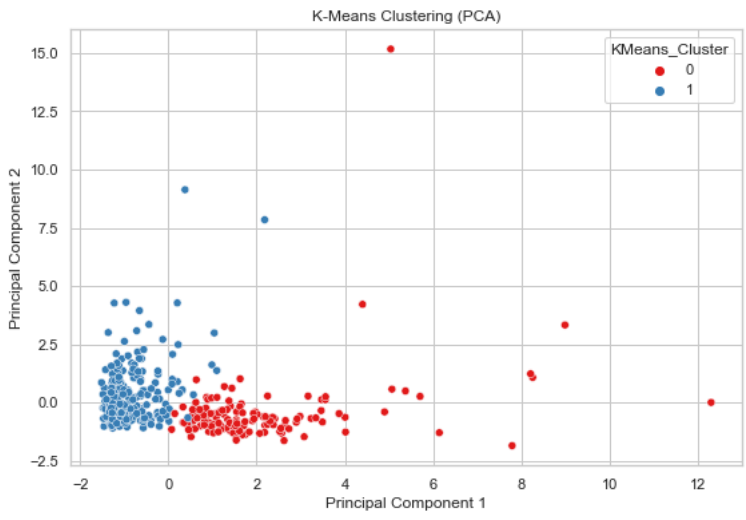



```
pca=PCA(n_components=2)
X_pca=pca.fit_transform(X_scaled)
print(
     "Variance explained by the teo PCA components:",
    round(pca.explained_variance_ratio_.sum()*100,2),
    "%"
)
plot_df=pd.DataFrame({"PC1":X_pca[:,0],"PC2":X_pca[:,1],
                      "KMeans_Cluster":kmeans_labels.astype(str),
                      "Hierarchical_cluster":hierarchical_labels.astype(str)
                      })
```

```
Variance explained by the teo PCA components: 61.12 %
```

```
plt.figure(figsize=(9,6))
sns.scatterplot(data=plot_df, x="PC1", y="PC2", hue="KMeans_Cluster", palette="Set1")

plt.title("K-Means Clustering (PCA)")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.show()
```




```
def evaluate_clustering(X_data,labels,model_name):
    return{
        "Algorithm":model_name,
        "Number of Clusters":len(np.unique(labels)),
        "Silhouette Score":silhouette_score(X_data,labels)
    }
```

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.